

3. Cache

3.1. 3 Konfigurationen fuer Cache Groesse 32 Words

Durch die Strategie *write-back*, wird das Datum bei einer write Operation zunaechst nur ins Cache geschrieben. Es wird spaetestens dann in den Hauptspeicher kopiert, wenn es aus dem Cache entfernt wird. *write-allocate* bedeutet hingegen, dass bei einem write-miss das Datum ins Cache geladen wird.

Moegliche Konfigurationen:

1. 4 Bloecke a 8 Words (direkte Abbildung)
2. 8 Bloecke a 4 Words (direkte Abbildung)
3. 4 Sets a 2 Bloecke a 4 Words (Assoziativitaetsgrad 2)

3.2 write allocate vs. write around

write around bedeutet, dass bei einem write-miss das Datum nicht ins Cache geschrieben wird. Die Strategie wird oft mit der write-through Strategie gepaart.

Beispiele fuer Situationen in denen *write around* sinnvoll ist sind generell Faelle, in denen zeitliche und raeumliche Lokalitaet nicht zutreffen: wenn wir nicht erwarten, dass unsere writes bald gelesen werden, waere die doppelte Schreibarbeit unter *write allocate* vergeudet und unser Cache mit nicht mehr gebrauchten Daten befuellt (Fachbegriff *cache pollution*). Konkreter: einmalige, verstreute schreib-Operationen auf eine Menge von Variablen, structs, oder sogar einen einzigen grossen Array.

Um Programmieren die Wahl zwischen den beiden Strategien zu ueberlassen, koennte man die *store* Instruktion in *store with write around* und *store with write allocate* verzweigen:

```
salw t0 0 t1 # (s)to(re) (w)ord using write (al)locate
sarw t0 0 t1 # (s)to(re) (w)ord using write (ar)ound
```

Standard C hat keinen Mechanismus, um eine per-store cache-policy auszudruecken. Denkt man sich entsprechende Befehle aus, so trifft man auf ein Problem:

```
int foo;

// Ein neuer Befehl fuer jede Policy:
// denkbar, aber man kommt in Schwierigkeiten, wenn man die Funktionssignatur
// definieren will.
//
// Versuch mit einem generic <T>type (existiert nicht in C!):
setal((int) &foo, 1); // int setal(<T> *dest, <T> val): write value to adress
// specified by argument `dest` using write-miss policy write allocate

setar((int) &foo, 2); // int setar(<T> *dest, <T> val): write value to adress
// specified by argument `dest` using write-miss policy write around

foo = 3; // standard syntax: let compiler decide

// Problem: Es laesst sich in C nicht leicht ausdruecken, dass der Typ von `val`
// derselbe wie der des Pointers `dest` ist.
// Wir braeuchten soetwas wie generic Types.
// Oder eine gesonderte Funktion fuer jeden Datentypen: `setal_int, setal_float,
// setal_char`, usw.
// Die Reibung zeigt, dass diese Modifikation problematisch waere.
```

Es ist auch aus einer weiteren Perspektive fraglich, diesen Mechanismus in dieser Weise in C-Code zu exponieren – es wird hier eine Abstraktionsgrenze ueberschritten. Angebrachter koennte es sein, entsprechende Compiler-Flags zu benutzen. Diese wuerden dann aber fuer das ganze translation unit gelten.

Fazit: einem C-Programm die Kontrolle ueber cache-policy write allocate vs. write around zu geben, ist einfacher gesagt als getan. Global liesse sich das per Compile-Flags erreichen. Aber um dies per store Anweisung zu kontrollieren, waeren erhebliche Aenderungen noetig (z.B. compiler extensions, oder intrinsics).

3.3 Strategie-Konfigurationen

Metrik:

$$\begin{pmatrix} \text{Hit Rate} & \text{Anzahl Writebacks} \\ \text{Anzahl Hits} & \text{Anzahl Misses} \end{pmatrix}$$

Wir waehlen diese Metrik in Anlehnung an die Moeglichkeiten der Ripes Simulation. Da wir primaer an der Laufzeit interessiert sind, aber Ripes nicht die cache access latency simuliert (siehe Dokumentation), nehmen wir die naechst-besten verfuegbaren Statistiken.

Offset	wb+wa	wt+wa	wb+nwa	wt+nwa
64	.78 0 14 4	.78 8 14 4	.44 8 8 10	.44 8 8 10
128	.44 0 8 10	.44 8 8 10	=	=

3.4 offset 128 und write allocate

Bei offset==128 werden das src und dest auf denselben Index gemapped. Fuer einen Cache mit Assoziativitaetsgrad 1 bedeutet das, dass sie auf denselben Index gemapped werden und um dieselbe Line konkurrieren. In der Schleife wird src per load gecached und dann gleich wieder von dest in der store Instruktion verdraengt. Das wiederholt sich und wir sehen entsprechende Verschlechterung in der Hit-Rate.

Begrueundung: Wir benutzen die Formel

$$\text{index} = (\text{address} \gg \text{offset_bits}) \bmod \text{number_of_lines}$$

Fuer uns ergibt das:

$$\text{index} = (\text{address} \gg 4) \bmod 2^3$$

Index wird also bestimmt durch Bits 4, 5, 6 der Adresse.

Untersuchen wir, was genau bei address + 64 bzw. address + 128, passiert sehen wir, dass wegen $64 = 2^6$; $128 = 2^7$ in dem einen Fall die Relevanten Bits veraendert werden, im anderen allerdings nicht. Resultat: die Speicherbloecke werden einmal verschiedenen Indizes zugeordnet, und einmal demselben.

Man koennte das Problem umgehen, indem man die Assoziativitaet auf 2 anhebt.– Dadurch koennten src und dest im selben Set gecached werden. Moegliche Konfiguration: 4 Sets a 2 Bloecke a 4 Words (Assoziativitaetsgrad 2).

3.5 Cache-Hits maximieren

Die zuwendende Idee ist, loads und stores nicht zu verflechten, sondern zu batchen: wir verwenden die ganze geladene Cache Line, erst danach gehen wir zur write-Phase ueber, in der die Line evtl. verdraengt wird. Verdraengung skaliert nun mit $n/\text{size cache line}$ und nicht mehr mit n .

Im Falle unseres Arrays, muessten wir einfach den loop ersetzen:

```
lw t3 0(t0)
lw t4 4(t0)
sw t3 0(t2)
sw t3 4(t2)
```

und kommen damit auf eine Hit-Rate von 67%.

Fuer eine skalierte Version bietet sich eine Schleife an, die mit ganzen Cache-Lines arbeitet:

```
// N = array length in words
// step of 4 -> our cache block size is 4 words
for i in 0..N-1 step 4:
    w0 = load word src[i+0]
    w1 = load word src[i+1]
    w2 = load word src[i+2]
    w3 = load word src[i+3]

    store word dest[i+0] = w0
    store word dest[i+1] = w1
    store word dest[i+2] = w2
    store word dest[i+3] = w3
```